

Install Required Libraries

```
pip install tensorflow numpy matplotlib
```

Import Libraries

```
import numpy as np
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    BatchNormalization,
    Dropout,
    Flatten,
    Dense
)
```

Task 1: Create a CNN Model with Conv2D Layer

```
model = Sequential()

model.add(
    Conv2D(
        filters=32,
        kernel_size=(3,3),
        activation="relu",
        input_shape=(32,32,3)
    )
)
```

Explanation

- **32 Filters** → Learn 32 different image features.
 - **3×3 Kernel** → Detects edges, corners, and textures.
 - **ReLU** → Adds non-linearity.
 - **Input Shape** → CIFAR-10 images are **32×32 RGB**.
-

Task 2: Add MaxPooling, Flatten, and Dense Layer

```
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
model.add(Flatten())
```

```
model.add(Dense(
    64,
    activation="relu"
))
```

Explanation

- **MaxPooling2D** reduces image size.
 - **Flatten** converts feature maps into a 1D vector.
 - **Dense(64)** learns higher-level image features.
-

Task 3: Add BatchNormalization and Dropout

```
model = Sequential([

    Conv2D(
        32,
        (3,3),
        activation="relu",
        input_shape=(32,32,3)
    ),

    BatchNormalization(),

    MaxPooling2D((2,2)),

    Flatten(),

    Dropout(0.3),

    Dense(
        64,
        activation="relu"
    ),

    Dense(
        10,
        activation="softmax"
    )
])
```

])

Explanation

BatchNormalization

- Normalizes activations.
- Speeds up training.
- Makes learning more stable.

Dropout(0.3)

- Randomly disables **30%** of neurons during training.
- Helps reduce overfitting.

Task 4: Compile and Train the Model

Compile Model

```
model.compile(  
  
    optimizer="adam",  
  
    loss="categorical_crossentropy",  
  
    metrics=["accuracy"]  
  
)
```

Generate Random CIFAR-10-Like Data

```
# 100 random images  
X = np.random.rand(  
    100,  
    32,  
    32,  
    3  
)  
  
# Random labels  
y = np.random.randint(  
    0,  
    10,
```

```
    100
)

# One-hot encoding
y = tf.keras.utils.to_categorical(
    y,
    num_classes=10
)
```

Train Model

```
history = model.fit(

    X,

    y,

    epochs=5,

    batch_size=16,

    validation_split=0.2

)
```

Print Model Summary

```
model.summary()
```

Example Output

Model: "sequential"

Layer (type)	Output Shape
--------------	--------------

=====

Conv2D	(None,30,30,32)
--------	-----------------

BatchNormalization	(None,30,30,32)
--------------------	-----------------

MaxPooling2D	(None,15,15,32)
--------------	-----------------

Flatten	(None,7200)
Dropout	(None,7200)
Dense	(None,64)
Dense	(None,10)

=====

Task 5: Improve the CNN Architecture

Suggested Improvement

Add another Conv2D + MaxPooling block before Flatten.

This allows the network to learn more complex features (edges → textures → object parts) before classification.

Improved CNN

```
model = Sequential([

    Conv2D(
        32,
        (3,3),
        activation="relu",
        input_shape=(32,32,3)
    ),

    BatchNormalization(),

    MaxPooling2D((2,2)),

    Conv2D(
        64,
        (3,3),
        activation="relu"
    ),

    BatchNormalization(),

    MaxPooling2D((2,2)),
```

```
Flatten(),

Dropout(0.3),

Dense(
    128,
    activation="relu"
),

Dense(
    10,
    activation="softmax"
)

])
```

Compile

```
model.compile(

    optimizer="adam",

    loss="categorical_crossentropy",

    metrics=["accuracy"]

)
```

Train

```
history = model.fit(

    X,

    y,

    epochs=5,

    batch_size=16,

    validation_split=0.2
```

)

Improved Architecture

Input Image (32×32×3)

|



Conv2D (32 Filters)

|



BatchNormalization

|



MaxPooling

|



Conv2D (64 Filters)

|

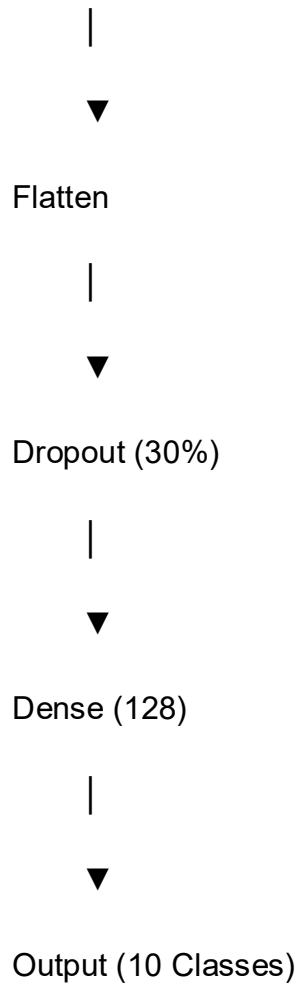


BatchNormalization

|



MaxPooling



Why is the Improved Model Better?

- The first convolutional layer learns **simple features** such as edges and corners.
- The second convolutional layer combines these into **more complex patterns**, such as textures and shapes.
- Batch Normalization stabilizes learning.
- Dropout reduces overfitting.
- A larger Dense layer provides greater capacity to learn meaningful feature combinations.